Assignment 1: Building a Packet Sniffer with libpcap

Course: Information Security (SENS-4533)

Total Marks: 100 (60 group assignment + 40 viva)

Due Date: on portal

Viva Date: will be declared in class after assignment submission

Member	Name	Roll No.	Module owned
1 (Group Leader)			Capture engine and integration
2			Ethernet dissection
3			IPv4 dissection
4			TCP/UDP dissection

Instructions for Solving This Assignment

Read every point below before you start. Marks are deducted for ignoring them.

1. **Form a group of exactly 4 members.**
2. **Divide the work, share the understanding.** Each member owns one module (see Group Organisation and Roles). Tasks 1 and 6 are done jointly. Every member must be able to run, explain and modify the whole program, not only their own module.
3. **One submission per group.** The Group Leader uploads it. All four names and roll numbers must appear on the report cover and in the header comment of `sniffer.c`.
4. **A viva will be conducted on this assignment (40 marks, individual).** Each member is questioned separately on the packet sniffer attack, libpcap and the group's code. Absence from the viva means zero viva marks. See the Viva Voce section.
5. **Language and library.** Write in C and use the libpcap API taught in the lecture (`pcap_lookupdev`, `pcap_open_live`, `pcap_loop`, and the callback function).
6. **Environment.** Use Linux (an Ubuntu virtual machine is recommended). Install the library with `sudo apt install libpcap-dev`. Compile with `gcc sniffer.c -o sniffer -lpcap`. Capturing needs privileges, so run with `sudo ./sniffer`.
7. **Ethics and legality (mandatory).** Capture traffic only on your own VM. Never capture other people's traffic. Before putting any output in your report, mask passwords, cookies, and personal data.
8. **Parse headers manually.** For Tasks 3 to 5 you must extract each field from the packet using byte offsets and shifts/masks, as in the lecture's `packet[0] ... packet[5]` example. You may NOT use `struct ether_header`, `struct ip`, `struct tcphdr`, or `struct udphdr`. Constants and the functions `ntohs()`, `ntohl()` and `inet_ntoa()` are allowed.
9. **Byte order.** Network headers are big-endian. Convert multi-byte fields (ports, lengths, sequence numbers, and so on) before printing.
10. **Build one program step by step.** Keep your code in a shared Git repository (or any shared folder with version history). Each member commits their own module, so the history shows who did what. Your report must show the output of each stage.

11. **Generating test traffic.** While the sniffer runs, create traffic from another terminal, for example `ping -c 3 8.8.8.8` (ICMP), `curl http://example.com` (TCP/HTTP), and `nslookup example.com` (UDP/DNS).
12. **Deprecated function note.** Newer libpcap versions mark `pcap_lookupdev()` as deprecated. If your compiler warns, the program still works. Optionally, accept the interface name as `argv[1]` (list interfaces with `ip link`) as a fallback.
13. **Comment your code.** Every function and every header field you decode needs a short comment. Put the member's name above each module.
14. **Academic integrity.** Collaboration is allowed inside your group only. Copying between groups gives zero to every member involved. Any code you cannot explain in the viva will be treated as not your own work.
15. **Submit** one ZIP named `GroupNo_A1.zip` containing (a) `sniffer.c`, (b) `report.pdf` including the contribution statement, and (c) screenshots referenced in the report. Late submissions lose 10% per day.

Group Organisation and Roles

Each member owns one module of the single `sniffer.c`; the group integrates them and answers Tasks 1 and 6 together.

Member	Module owned	Tasks
1 (Group Leader)	Capture engine and integration: interface selection, promiscuous capture loop, packet counter, merging all modules	Task 2
2	Ethernet header dissection and frame-type counters	Task 3
3	IPv4 header dissection	Task 4
4	TCP/UDP dissection and port-to-application mapping	Task 5
All members	Concept answers, manual packet decoding, report	Tasks 1, 6,

- Roles may be swapped by agreement, but the final roles must be written in the contribution statement.
- Each member reviews the module of the next member (1 reviews 2, 2 reviews 3, 3 reviews 4, 4 reviews 1) and notes one improvement in the report.
- The contribution statement lists each member, the module owned, the tasks completed and an effort percentage, signed by all four.

Quick Reference (from the lecture)

Function	Purpose
`char* pcap_lookupdev(char* errbuf)`	Returns the name of the first network device suitable for capture. `errbuf` must hold at least `PCAP_ERRBUF_SIZE` bytes.
`pcap_t* pcap_open_live(device, snaplen, promisc, to_ms, errbuf)`	Opens the interface. `snaplen` is the max bytes captured, `promisc` is 0 for non-promiscuous (any other value for promiscuous), `to_ms` is the wait before copying data from kernel to user space.

Function	Purpose
`u_char* pcap_next(pcap_t* p, struct pcap_pkthdr* h)`	Captures and returns one packet.
`int pcap_loop(p, count, callback, user)`	Captures continuously and calls your callback per packet. A negative `count` means capture until an error occurs.
`void callback(u_char* userarg, const struct pcap_pkthdr* pkthdr, const u_char* packet)`	Your packet-processing function. `pkthdr` holds `ts`, `caplen`, `len`.

Starter Code

```c
/* Compile: gcc sniffer.c -o sniffer -lpcap      Run: sudo ./sniffer */
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>   /* ntohs(), ntohl(), inet_ntoa() */

#define MAXBYTES 2048

void process_pkt(u_char *args, const struct pcap_pkthdr *pkthdr, const u_char *packet);

int main(int argc, char **argv)
{
    char errbuff[PCAP_ERRBUF_SIZE];
    pcap_t *descptr = NULL;
    int count = 0;

    /* TODO 1: get the device name (pcap_lookupdev, or argv[1]) */
    /* TODO 2: open it in promiscuous mode (pcap_open_live) */
    /* TODO 3: capture continuously (pcap_loop) with process_pkt as callback */
    /* TODO 4: print summary statistics, then pcap_close() */
    return 0;
}

void process_pkt(u_char *args, const struct pcap_pkthdr *pkthdr, const u_char *packet)
{
    /* TODO: dissect Ethernet -> IPv4 -> TCP/UDP here */
}
```

Task 1: Concept Check (20 marks)

Answer in 2 to 4 sentences each, in your own words, in the report.

1. Describe the steps of normal packet processing from the NIC to the application. At which step does a NIC in **promiscuous mode** behave differently? **(4 marks)**
2. Why is it inefficient for the packet filter to send every packet to user space? How does **kernel-level filtering** help? **(4 marks)**
3. How does the OS decide which protocol handler receives a frame (use the Ether Type field) and which upper-layer protocol receives an IP payload (use the Protocol field)? How does TCP/UDP then find the destination application? **(4 marks)**
4. Explain the roles of the `snaplen`, `promisc`, and `to_ms` parameters of `pcap_open_live()`. **(4 marks)**
5. Give two legitimate and two malicious uses of a packet sniffer. **(4 marks)**

Task 2: Basic Packet Capture (15 marks)

Complete the TODOs in the starter code so that the program:

1. finds a network interface and prints its name **(3 marks)**
2. opens it in promiscuous mode with `MAXBYTES` as snaplen, handling errors by printing `errbuff` and exiting **(4 marks)**
3. captures packets with `pcap_loop()` and, in the callback, increments a counter passed through the `user` argument **(4 marks)**
4. for every packet prints the packet number, capture timestamp, `caplen` and `len` **(4 marks).**

Example output format:

```
Packet #1  time: 1760000000.123456  caplen: 74  len: 74
```

Task 3: Ethernet Header Dissection (15 marks)

Extend the callback to decode the Ethernet header (first 14 bytes):

1. Print the destination and source MAC addresses in the form `AA:BB:CC:DD:EE:FF` **(5 marks).**
2. Read the 2-byte Type field and print its meaning: `0x0800` IPv4, `0x0806` ARP, `0x86DD` IPv6, otherwise Other **(5 marks).**
3. Keep running counts of IPv4, ARP, IPv6 and Other frames, and print them when the capture ends. For example, stop after 50 packets by passing `count = 50` to `pcap_loop()` **(5 marks).**

Task 4: IPv4 Header Dissection (20 marks)

For frames whose Type is `0x0800`, decode the IPv4 header that follows the Ethernet header. Print all of the following:

Field	What to print
Version	Value (should be 4)
IHL	Value in 32-bit words **and** the header length in bytes (IHL x 4)
Total Length	Datagram length in bytes
Identification	Value in hex
Flags	Whether Don't Fragment (DF) and More Fragments (MF) are set
Fragment Offset	Value and the byte offset (value x 8)
Time to Live	Value
Protocol	Number and name (6 = TCP, 17 = UDP, others = Other)
Header Checksum	Value in hex
Source / Destination IP	Dotted-decimal form

Marks distribution for the task 4 :

1. Version, IHL and Total Length **(6 marks)**
2. Identification, Flags and Fragment Offset **(6 marks)**

3. TTL, Protocol and Checksum **(4 marks)**
4. IP addresses **(4 marks).**

Hint: the IHL field tells you where the transport header starts. Do not assume the IP header is always 20 bytes, because options can make it longer.

Task 5: TCP and UDP Header Dissection (20 marks)

Using the Protocol field, jump to the transport header (start = 14 + IHL x 4) and decode:

1. **TCP:** source port, destination port, sequence number, acknowledgement number, header length (in words and bytes), and all six flags (URG, ACK, PSH, RST, SYN, FIN), printing the names of the flags that are set (for example `SYN ACK`). **(12 marks)**
2. **UDP** source port, destination port, length, and checksum. **(4 marks)**
3. **Application guess:** use the well-known port numbers from the lecture (web 80, email 25, FTP 20/21) and print a likely application name when either port matches. Extend the list with at least three more ports of your choice (for example DNS 53, HTTPS 443, SSH 22). **(4 marks)**

Task 6: Manual Packet Decoding (10 marks)

A sniffer captured the following 54-byte frame (hex). **Without running your program**, decode it by hand in the report, showing the byte positions you used.

```
00 1A 2B 3C 4D 5E 08 00 27 AA BB CC 08 00
45 00 00 28 1C 46 40 00 40 06 26 FD C0 A8 01 0A 5D B8 D8 22
C3 50 00 50 00 00 03 E8 00 00 00 00 50 02 72 10 1A 2B 00 00
```

Answer the following:

1. Destination MAC, source MAC and the upper-layer protocol indicated by the Ether Type. **(2 marks)**
2. IP version, header length in bytes, total length, TTL, protocol, and source and destination IP addresses. **(3 marks)**
3. Are the DF or MF flags set? What is the fragment offset? **(1 marks)**
4. Source port, destination port, sequence number, TCP header length, and which flags are set. **(3 marks)**
5. What is the purpose of this packet within a TCP connection? Which application is the destination port associated with? **(1 marks)**

Then run your own program on this frame's byte values or on a real capture and confirm the decoded fields match your manual work (screenshot optional).

Report Requirements

Your `report.pdf` must contain, in order: (1) a cover page with the group number and the names and roll numbers of all four members, (2) answers to Task 1, (3) for each of Tasks 2 to 5 a screenshot of the program output plus two or three sentences describing what you observed, (4) the manual decoding for Task 6, (5) the bonus explanation if attempted, and (6) the signed contribution statement and peer-review notes. Reference screenshots by task number.